



Nohup over SSH

Last updated: April 11, 2023 | [Linuxopsys](#)

Contents



All running Linux processes terminate when we exit the shell. If we want to keep those processes running despite exiting the terminal we take the help of the nohup command-line utility. Prefixing a command with nohup prevents it from being aborted automatically.

In this tutorial, we learn about how to use nohup over ssh.

1. Running Remote Command over SSH

SSH stands for Secure Shell. It is a secure network protocol for communicating with a remote server. SSH authenticates both the parties by employing a client-server model, and passes commands and output from source to destination and vice versa. The data is encrypted between them.

Syntax:

```
ssh -p <PortNumber> username@host 'command1 && command2'
```

The host can either be a domain name or an IP address. The default port number is 22

Let's try this example:

```
ssh ubuntu@ip-address-or-servername 'pwd && date'
```

Please make sure to pass the username and hostname as per your environment.

After making the connection to the remote machine, the instructions or the commands given along with the [SSH](#) command, will work as if they were meant directly to this host terminal.

However, once this SSH connection gets closed the commands, which are executing, get killed. When we SSH, its daemon allocates us a terminal. When this daemon finds our connection dead, it releases a SIGHUP signal to the terminal, notifying that the connection is dead. It will then begin issuing the SIGHUP signal to the running processes associated with it before terminating. All these processes will terminate upon receiving this signal.

```
ubuntu@linuxopsys:~/script/30-03-2023$ ssh -p 33 ubuntu@37.135.126.20 "pwd && date"
The authenticity of host '[37.135.126.20]:33 ([37.135.126.20]:33)' can't be established.
ECDSA key fingerprint is SHA256:N+B7nPwngfjzGEt8klIfws6lPIiaxDM2BHUo+5Z
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[37.135.126.20]:33' (ECDSA) to the list of known hosts.
ubuntu@37.135.126.20's password:
/home/ubuntu
Mon Apr 10 07:09:51 UTC 2023
ubuntu@linuxopsys:~/script/30-03-2023$
```

We are securely connecting to the server using the ssh command to run multiple commands on the remote machine. The default port is 22. Once in, we are executing the pwd command on this server to fetch the current working directory along with the date command.

2. Nohup over SSH

A background process is a process that starts its execution from the terminal and runs in the background. It doesn't have any interaction with the user. This process runs independent of the shell. We can use the nohup command to make a script or a command work in the background.

Nohup stands for No Hang Up. It enables processes to run even after exiting the shell. This is achieved by intercepting and blocking the processes from receiving the SIGHUP (Signal Hang Up) signal. This signal is sent to a process upon exiting the terminal.

Here's an example:

```
nohup ./loopRange2.sh &
```

The ampersand symbol (&) symbol is incorporated at the end of the command which we want to run in the background. After the execution of the process running in the background finishes, it doesn't return to the terminal. It can be made available in the foreground with the use of the fg command.

```
ubuntu@linuxopsys:~/script/30-03-2023$ ./loopRange2.sh
Forward
1
3
5
7
9
Reverse
10
8
6
4
2
ubuntu@linuxopsys:~/script/30-03-2023$ nohup ./loopRange2.sh &
[1] 2218
ubuntu@linuxopsys:~/script/30-03-2023$ nohup: ignoring input and appending output to 'nohup.out'

[1]+  Done                  nohup ./loopRange2.sh
ubuntu@linuxopsys:~/script/30-03-2023$
```

Using this command we are running the script in the background. The output generated on the screen are numbers. The first number, within square brackets, corresponds to the job id whereas the next one represents the process id.

The same could be used to do a secure connection and run the command in the background. The nohup command coupled **with SSH** ensures that the scripts continue to execute over SSH even if the remote session is terminated or there is a network drop.

Example:

Let us assume we have a script snooze.sh with just a sleep command of 1000 seconds.

```
echo "Hi"
sleep 1000
```

Let's try to execute this script remotely with a single line SSH command:

```
ssh ubuntu@ip-address-or-servername "nohup ./snooze.sh &"
```

As we can see, the script is running but the ssh connection is still open until it sleeps for a thousand seconds. To launch remotely, and immediately return we need to make sure the input and output are not open.

SSH with nohup will be as follows:

```
ssh -n ubuntu@ip-address-or-servername "nohup ./snooze.sh > /dev/
```

```
ubuntu@linuxopsys:~/script/30-03-2023$ ssh -p 33 ubuntu@ 37.135.126.20 "cat snooze.sh"
ubuntu@ 37.135.126.20 's password:
echo Hi
sleep 1000
ubuntu@linuxopsys:~/script/30-03-2023$ ssh -p 33 ubuntu@ 37.135.126.20 "nohup ./snooze.sh &"
ubuntu@ 37.135.126.20 's password:
Hi
ubuntu@linuxopsys:~/script/30-03-2023$
ubuntu@linuxopsys:~/script/30-03-2023$
ubuntu@linuxopsys:~/script/30-03-2023$
ubuntu@linuxopsys:~/script/30-03-2023$ ssh -p 33 ubuntu@ 37.135.126.20 "nohup ./snooze.sh > /dev/null 2>&1 &"
ubuntu@ 37.135.126.20 's password:
ubuntu@linuxopsys:~/script/30-03-2023$ █
```

This command executes the script "snooze" on the remote system. Even after the termination of the ssh command, the snooze script continues to run in the background. We will learn how to verify the script is running, despite the session termination, in the next section. We can use ssh with the optional -n option to prevent reading from the stdin. In our case, even without the -n flag, the control will instantaneously return to the terminal.

Here's another example:

```
nohup scp somefile ubuntu@ip-address-or-servername:/home/ubuntu/I
```

This ensures that this command runs in the background. We are doing a Secure Copy (SCP) to transfer the file to that system. The 2>&1 redirection moves the standard error to wherever the standard output is being redirected as well. If we want the console logs, we can redirect them to a file. In this case we are sending it to a special file named '/dev/null'. This file is considered a blackhole. Anything can be sent to this file and nothing can be retrieved from it.

```

ubuntu@linuxopsys:~/script/30-03-2023$ scp -P 33 top.sh ubuntu@20.172.152.27:/home/ubuntu/Downloads
ubuntu@20.172.152.27's password:
top.sh 100% 32 1.2KB/s 00:00
ubuntu@linuxopsys:~/script/30-03-2023$ nohup scp -P 33 top.sh ubuntu@20.172.152.27:/home/ubuntu/Downloads /dev/null 2>&1 &
[1] 2100
ubuntu@linuxopsys:~/script/30-03-2023$ nohup: ignoring input and appending output to 'nohup.out'

[1]+ Exit 1 nohup scp -P 3399 top.sh ubuntu@20.172.152.27:/home/ubuntu/Downloads /dev/null 2>&1
ubuntu@linuxopsys:~/script/30-03-2023$

```

We can disown the job as well so that it is disjoint from the shell session. We can get the corresponding job id using the jobs command with the -l option. Once the job is identified we can disown it.

```

jobs -l
disown %1

```

```

ubuntu@linuxopsys:~/script/30-03-2023$ jobs -l
[1]+ 3883 Running nohup ./top.sh &
ubuntu@linuxopsys:~/script/30-03-2023$
ubuntu@linuxopsys:~/script/30-03-2023$
ubuntu@linuxopsys:~/script/30-03-2023$ disown %1
ubuntu@linuxopsys:~/script/30-03-2023$ jobs -l
ubuntu@linuxopsys:~/script/30-03-2023$

```

The Linux disown command is used to remove jobs from the job table. It also instructs the shell not to send a HUP signal to the active jobs. This command is useful to remove the jobs or keep them running even after logging off from the shell session.

3. Checking the Status

Once the command or a script is running as a background process, we can find its status.

Here's an example:

```

ps -ef

```

This will list all the processes on this system in a detailed fashion. We can filter the results by making use of the Bash utility grep, along with the Linux pipe as follows:

```
ps -ef | grep <PID>
```

The `ps` command is a handy tool that lists all the processes which are running along with their PIDs (Process IDs) and PPIDs (Parent Process ID). The `-e` flag, standing for entire, displays all the processes within the system and is not just restricted to the current terminal. The `-f` option gives a detailed view of the processes.

How to check the status of a script that ran as a background with Nohup?

We can get the PID by writing it to a file at the beginning of our shell script. Once we have the file with the PID, we can simply search for that PID to know if the process is done executing or not. To record the PID, we can add the following to our script:

```
echo $$ > processIDFile.pid
```

Once we have the PID of the background process, we can stop it by providing this PID to the `kill` command.

```
kill -9 <PID>
```

How to check the status of a remote script that ran as a background with Nohup?

Putting all this together, we can now SSH to a remote server as we did earlier and execute these commands.

```
ssh ubuntu@ip-address-or-servername 'nohup ./top.sh &'
ssh ubuntu@ip-address-or-servername 'cat PID.pid'
ssh ubuntu@ip-address-or-servername 'ps -ef | grep <PID>'
ssh ubuntu@ip-address-or-servername 'kill -9 <PID>'
ssh ubuntu@ip-address-or-servername 'ps -ef | grep <PID>'
```

```

ubuntu@linuxopsys:~/script/30-03-2023$ ssh -p 33 ubuntu @ 37.135.126.20 "cat snooze.sh"
ubuntu @ 37.135.126.203's password:
echo $$ > processIDFile.pid
echo Hi
sleep 1000
ubuntu@linuxopsys:~/script/30-03-2023$ ssh -p 33 ubuntu @ 37.135.126.20 "nohup ./snooze.sh > /dev/null 2>&1 &"
ubuntu @ 37.135.126.20 's password:
ubuntu@linuxopsys:~/script/30-03-2023$ ssh -p 33 ubuntu @ 37.135.126.20 "cat processIDFile.pid"
ubuntu @ 37.135.126.20 's password:
4819
ubuntu@linuxopsys:~/script/30-03-2023$ ssh -p 33 ubuntu @ 37.135.126.20 "ps -ef | grep 4819"
ubuntu @ 37.135.126.203's password:
ubuntu      4819      1   0  07:32 ?        00:00:00 /bin/sh ./snooze.sh
ubuntu      4820     4819   0  07:32 ?        00:00:00 sleep 1000
ubuntu      4933    1443   0  07:33 pts/0    00:00:00 ssh -p 33 ubuntu @ 37.135.126.20 ps -ef | grep 4819
ubuntu      5015    5014   0  07:33 ?        00:00:00 bash -c ps -ef | grep 4819
ubuntu      5017    5015   0  07:33 ?        00:00:00 grep 4819
ubuntu@linuxopsys:~/script/30-03-2023$ ssh -p 33 ubuntu @ 37.135.126.20 "kill -9 4819"
ubuntu @ 37.135.126.203's password:
ubuntu@linuxopsys:~/script/30-03-2023$ ssh -p 33 ubuntu @ 37.135.126.20 "ps -ef | grep 4819"
ubuntu @ 37.135.126.203's password:
ubuntu      5125    1443   0  07:33 pts/0    00:00:00 ssh -p 33 ubuntu @ 37.135.126.20 ps -ef | grep 4819
ubuntu      5208    5207   0  07:33 ?        00:00:00 bash -c ps -ef | grep 4819
ubuntu      5210    5208   0  07:33 ?        00:00:00 grep 4819
ubuntu@linuxopsys:~/script/30-03-2023$ █

```

We are listing the PID of the background process. If it is a script, we can add the PID to some file and read it. Once we get the file, we are terminating the script or the process corresponding to that PID using the kill command.

Conclusion

- SSH establishes a secure connection to the remote server.
- The nohup command allows us to keep the process/script/command running despite exiting the terminal by intercepting the SIGHUP command.
- The ps command with its flags can be used to find the status and kill the background process.

If this resource helped you, let us know your care by a Thanks Tweet.

[Tweet a thanks](#)

Did you find this article helpful?



[How to Create a Certificate Authority \(CA\) on Ubuntu](#)

[Set up SmartCard Authentication on Linux](#)

[How to Disable Root Login for SSH](#)

[chroot Directory with Example](#)

Comments

Please add comments below to provide the author your ideas, appreciation and feedback.

Leave a Reply.

Post Comment

***Wrap code samples within <pre>...</pre> tag to ensure the format remains intact

Contents



1. Running Remote Command over SSH

2. Nohup over SSH

3. Checking the Status

Conclusion

Related Posts

May 04, 2023

chroot Directory with Example

Mar 13, 2024

How to Add User to Sudoers File or Sudo Group on Ubuntu

Jul 18, 2022

How to Enable SSH on Debian 11

Quick Links

[About](#) | [Contact](#)

[Terms of Service](#) | [Privacy Policy](#)

© 2022 - 2025 LinuxOPsys